```
) ;
```

شکل ۳-۱۸ برنامه‌ی نمونه‌ای که عبور پیغام را در Mach نشان می‌دهد.

گزینه‌ی آخر در رابطه با وظایف کارگزار است. وظیفه‌ی کارگزار بعد از پایان یک درخواست، نیاز دارد که پاسخی یک‌باره به وظیفه‌ی درخواست‌کننده‌ی سرویس ارسال کند، اما بایستی به سرویس درخواست‌های دیگر ادامه بدهد، حتی اگر پورت پاسخ کارفرما پر باشد.

مسأله‌ی اصلی که کلاً در سیستم‌های پیغام وجود دارد، کارایی ضعیفی است که از کپی مضاعف پیغام‌ها از پورت فرستنده در پورت گیرنده ناشی می‌شود. سیستم پیغام Mach در تلاش است تا با استفاده از تکنیک‌های مدیریت حافظه‌ی مجازی (فصل ۱۰) از عملیات کپی اجتناب کند. Mach در اصل، فضای آدرس مربوط به پیغام فرستنده را به فضای آدرس گیرنده نگاشت می‌دهد. بنابراین، خود پیغام به واقع هرگز کپی نمی‌شود، چون هر دوی فرستنده و گیرنده حافظه‌ی مشترکی را دست‌یابی می‌کنند. این تکنیک مدیریت پیغام، کمک زیادی به بالارفتن کارایی می‌نماید، اما کارکرد آن فقط در پیغام‌های داخلی سیستم است.

۳-۷-۳ ویندوز

سیستم‌عامل ویندوز، مثالی از یک طراحی مدرن است که برای افزایش کارکرد و کاهش زمان لازم برای پیاده‌سازی امکانات جدید، ماژولاریتی را بکار می‌برد. ویندوز، پشتیبانی محیط‌های عملیاتی متعدد یا *زیرسیستم‌ها* را فراهم می‌کند. ارتباط برنامه‌های کاربردی با این زیرسیستم‌ها از طریق یک راهکار عبور پیغام صورت می‌گیرد. بنابراین، می‌توان برنامه‌های کاربردی را کارفرماهای یک کارگزار زیرسیستم در نظر گرفت.

امکان عبور پیغام ویندوز، امکان **فراخوانی روال محلی پیشرفته**[1] (**ALPC**) نامیده می‌شود که برای ارتباط میان دو پردازه روی ماشین یکسان استفاده می‌شود. آن مشابه راهکار معمول فراخوانی روال دور (RPC) است که به صورت گسترده استفاده می‌شود، اما بهینه شده و خاص ویندوز می‌باشد. (فراخوانی‌های روال دور با جزئیات بیشتر در بخش ۳-۸-۲ پوشش داده می‌شوند.) ویندوز هم مثل Mach، برای برپایی و حفظ یک اتصال میان دو پردازه از یک شئ درگاه استفاده می‌کند. ویندوز از دو نوع درگاه استفاده می‌کند که عبارتند از: **درگاه‌های اتصال**[2] و **درگاه‌های ارتباط**[3].

پردازه‌های کارگزار، اشیاء درگاه اتصال را منتشر می‌کنند که برای تمامی پردازه‌ها قابل رؤیت هستند. موقعی که یک کارفرما سرویس‌هایی را از یک زیرسیستم می‌خواهد، دستگیره‌ای به شئ درگاه اتصال کارگزار باز کرده و یک درخواست اتصال به آن درگاه می‌فرستند. سپس، کارگزار یک کانال ایجاد کرده و یک دستگیره به کارفرما بر می‌گرداند. کانال، حاوی یک جفت درگاه ارتباط خصوصی است: یکی برای پیغام‌های کارفرما–کارگزار، دیگری برای پیغام‌های

1. Advanced local procedure call
2. Connection ports
3. Communication ports

کارگزار–کارفرما. به علاوه، کانال‌های ارتباطی از یک راهکار بازگشت صدا پشتیبانی می‌کنند که به کارفرما و کارگزار امکان می‌دهد، درخواست‌ها را موقعی بپذیرند که علی‌القاعده انتظار یک پاسخ را دارند.

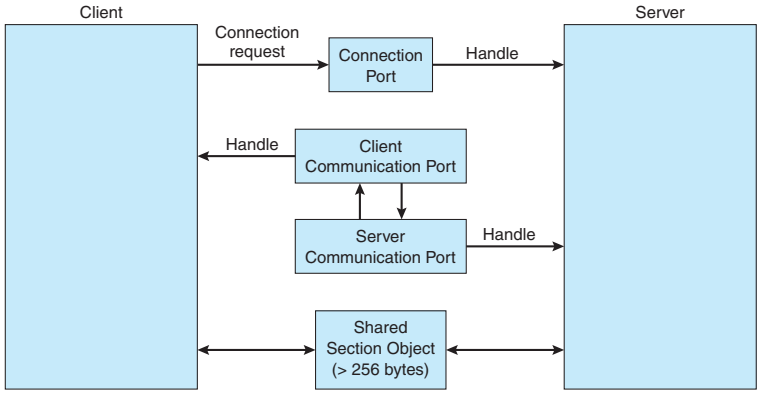
یکی از سه تکنیک عبور پیغام زیر در موقع ایجاد یک کانال ALPC انتخاب می‌شود:

۱- برای پیغام‌های کوچک (تا ۲۵۶ بایت) از صف پیغام درگاه به عنوان حافظه‌ی میانی استفاده می‌شود و پیغام‌ها از یک پردازه به دیگری کپی می‌شوند.

۲- پیغام‌های بزرگ‌تر بایستی از طریق یک شئ **بخش**[1] عبور داده شوند که ناحیه‌ای از حافظه‌ی مشترک مرتبط با کانال است.

۳- موقعی که حجم داده‌ها خیلی بزرگ است که در یک شئ بخش نمی‌گنجد، یک API وجود دارد که به پردازه‌های کارگزار امکان خواندن و نوشتن مستقیم در فضای آدرس یک کارفرما را فراهم می‌کند.

کارفرما به ناچار بایستی تصمیم بگیرد که چه موقع کانال را برپا می‌کند جدا از این که، نیاز به ارسال یک پیغام بزرگ خواهد داشت و یا خیر. اگر کارفرما دریابد که می‌خواهد پیغام‌های بزرگ ارسال کند، درخواست می‌دهد که یک شئ بخش ایجاد شود. به طور مشابه، اگر کارگزار ببیند که پاسخ‌ها بزرگ خواهند بود، یک پیغام کوچک که شامل یک اشاره‌گر و اطلاعات اندازه‌ی مربوط به شئ بخش است ارسال می‌کند. این روش، پیچیده‌تر از روشی است که در بالا عنوان شد، اما از کپی داده‌ها اجتناب می‌ورزد. ساختار فراخوانی‌های روال محلی پیشرفته در ویندوز در شکل ۳-۱۹ نشان داده می‌شود.



شکل ۳-۱۹ فراخوانی‌های روال محلی پیشرفته در ویندوز.

توجه به این نکته مهم است که امکان ALPC در ویندوز به عنوان بخشی از API ویندوز محسوب نمی‌شود و از این رو، برای برنامه‌نویس کاربردی قابل رؤیت نیست. به جای آن، برنامه‌های کاربردی که از API ویندوز استفاده می‌کنند، فراخوانی‌های روال دور معمول را

1. Section object

احضار می‌کنند. موقعی که RPC روی پردازه‌ای از سیستم یکسان احضار می‌شود، RPC به صورت غیرمستقیم از طریق یک فراخوانی روال ALPC راه‌اندازی می‌شود. به علاوه، بسیاری از سرویس‌های هسته برای ارتباط با پردازه‌های کارفرما از ALPC استفاده می‌کنند.

۳-۷-۴ لوله‌ها

یک **لوله** به‌عنوان مجرایی عمل می‌کند که اجازه می‌دهد دو پردازه با هم ارتباط داشته باشند. لوله‌ها یکی اولین راهکارهای IPC در سیستم‌های یونیکس اولیه بودند. آن‌ها نوعاً یکی از روش‌های ساده‌تر ارتباط پردازه‌ها با همدیگر را فراهم می‌کنند، اگرچه یک سری محدودیت هم دارند. در پیاده‌سازی یک لوله بایستی چهار موضوع مد نظر قرار بگیرند:

۱- آیا لوله ارتباط دوطرفه را اجازه می‌دهد، یا ارتباط یک‌طرفه است؟
۲- اگر ارتباط دوطرفه مجاز است، آیا نیمه‌دوطرفه است (داده‌ها در هر لحظه می‌توانند تنها در یک راه باشند) یا تمام دوطرفه (داده‌ها می‌توانند همزمان در هر دو جهت گردش داشته باشند)؟
۳- آیا بایستی یک رابطه‌ای (همچون **پدر-فرزندی**) میان پردازه‌های ارتباط وجود داشته باشد؟
۴- آیا لوله‌ها در یک شبکه ارتباط برقرار می‌کنند، یا بایستی پردازه‌های ارتباط روی ماشین یکسانی قرار داشته باشند؟

در بخش‌های بعدی، ما دو نوع لوله را که در هر دوی سیستم‌های یونیکس و ویندوز استفاده می‌شوند بررسی می‌کنیم که عبارتند از: لوله‌های معمولی و لوله‌های با نام.

۳-۷-۴-۱ لوله‌های معمولی

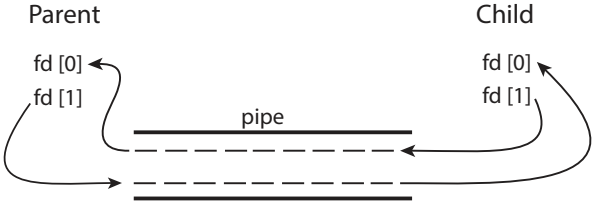
لوله‌های معمولی، امکان ارتباط دو پردازه را به شیوه‌ی معمول تولیدکننده-مصرف‌کننده فراهم می‌کنند؛ تولیدکننده از یک طرف در لوله می‌نویسد (**طرف نوشتن**) و مصرف‌کننده نیز از طرف دیگر می‌خواند (**طرف خواندن**). در نتیجه، لوله‌های معمولی یک طرفه هستند که فقط ارتباط یک سویه را فراهم می‌نمایند. اگر ارتباط دو سویه لازم شود، بایستی از دو لوله استفاده شود که هر لوله داده‌ها را در جهتی متفاوت ارسال می‌نماید. در ادامه، ساخت لوله‌های معمولی را در هر دوی سیستم‌های یونیکس و ویندوز نشان می‌دهیم. در هر دو مثال برنامه‌ها، یک پردازه پیغام «Greetings» را در لوله می‌نویسد، در حالی که پردازه‌ی دیگر، این پیغام را از لوله می‌خواند.

در سیستم‌های یونیکس، لوله‌های معمولی با استفاده از تابع زیر بنا می‌شوند:

```
pipe(int fd[])
```

این تابع یک لوله ایجاد می‌کند که از طریق توصیف‌گرهای فایل `fd[]` دستیابی می‌شود: `fd[0]` طرف خواندن لوله و `fd[1]` طرف نوشتن لوله می‌باشد. یونیکس با یک لوله همانند نوع خاصی از فایل رفتار می‌کند. بنابراین، لوله‌ها می‌توانند با استفاده از فراخوانی‌های سیستمی معمولی read() و write() دستیابی شوند.

یک لوله‌ی معمولی نمی‌تواند خارج از پردازه‌ای که آن را ایجاد می‌کند، مورد دسترسی قرار بگیرد. نوعاً یک پردازه‌ی والد، لوله‌ای را ایجاد می‌کند و با استفاده از آن با فرزندی که از طریق ()fork ایجاد کرده، ارتباط برقرار می‌نماید. از بخش ۳-۳-۱ به خاطر دارید که پردازه‌ی فرزند، فایل‌های باز والدش را به ارث می‌برد. از آنجایی که، یک لوله، نوع خاصی از فایل است، فرزند لوله را از پردازه‌ی والد خود به ارث می‌برد. شکل ۳-۲۰، رابطه‌ی توصیف‌گرهای فایل را در آرایه‌ی fd با پردازه‌های والد و فرزند نشان می‌دهد. همان‌گونه که روشن است، هرگونه نوشتنی توسط والد در طرف نوشتن لوله—[1]fd—می‌تواند توسط فرزند از طرف خواندن آن لوله—[0]fd—خوانده شود.



شکل ۳-۲۰ توصیف‌گرهای یک لوله‌ی معمولی.

در برنامه‌ی یونیکس نشان داده شده در شکل ۳-۲۱، پردازه‌ی والد لوله‌ای را ایجاد کرده و سپس به منظور ایجاد یک پردازه‌ی فرزند یک فراخوانی ()fork صادر می‌کند. این که بعد از فراخوانی ()fork چه اتفاقی می‌افتد بستگی به چگونگی جریان داده‌ها در لوله دارد. در اینجا، والد در لوله می‌نویسد و فرزند از آن می‌خواند. توجه به این نکته مهم است که هر دوی پردازه‌های والد و فرزند، در ابتدا طرف‌های بلااستفاده‌ی لوله را می‌بندند. اگرچه در برنامه‌ای که در شکل ۳-۲۱ نشان داده می‌شود به این عمل نیازی نیست، یکی از گام‌های مهم آن است که مطمئن شویم پردازه‌ای که از لوله می‌خواند انتهای فایل را می‌تواند، موقعی که نویسنده طرف لوله‌ی خود را بسته است، تشخیص دهد (()read، ۰ بر می‌گرداند).

از لوله‌های معمولی در سیستم‌های ویندوز تحت عنوان **لوله‌های بی‌نام** یاد شده و رفتاری شبیه به متناظرشان در یونیکس دارند: آنها یک‌طرفه بوده و ارتباطات والد-فرزندی را میان پردازه‌های مرتبط به کار می‌برند. به علاوه، خواندن و نوشتن لوله می‌تواند با توابع معمولی ()ReadFile و ()WriteFile انجام شود. API ویندوز جهت ایجاد لوله‌ها تابع ()CreatePipe است که چهار پارامتر به آن ارسال می‌شود. پارامترها دستگیره‌های مجزایی برای (۱) خواندن و (۲) نوشتن لوله، در کنار (۳) نمونه‌ای از رکورد STARTUPINFO که استفاده می‌شود تا مشخص کند قرار است پردازه‌ی فرزند دستگیره‌های لوله را به ارث بگیرد. مضاف بر این، (۴) اندازه‌ی لوله (برحسب بایت) می‌تواند مشخص شود.

شکل ۳-۲۳، پردازه‌ی والدی را نشان می‌دهد که یک لوله‌ی بی‌نام را جهت ارتباط با فرزندش ایجاد می‌کند. برعکس سیستم‌های یونیکس که پردازه‌ی فرزند به صورت خودکار، لوله‌ی ایجاد شده توسط والدش را به ارث می‌برد، ویندوز برنامه‌نویس را ملزم می‌سازد که صفاتی را که قرار

است پردازه‌ی فرزند به ارث ببرد، مشخص کند. این کار، نخست با مقداردهی آغازین رکورد SECURITY_ATTRIBUTES انجام می‌شود که امکان ارث‌بری دستگیره‌ها را ممکن می‌سازد و پس از آن، دستگیره‌های پردازه‌ی فرزند برای ورودی یا خروجی معمول به دستگیره‌ی خواندن یا نوشتن لوله هدایت می‌شوند. چون فرزند از لوله خواهد خواند، بایستی والد ورودی معمول فرزند را به دستگیره‌ی خواندن لوله هدایت کند. به علاوه، چون لوله‌ها نیمه دوطرفه هستند، لازم است که فرزند از ارث‌بری طرف نوشتن لوله منع شود. برنامه‌ی ایجاد پردازه‌ی فرزند مشابه برنامه‌ی شکل ۳-۱۰ است، جز این که، پارامتر پنجم، مقدار TRUE انتخاب می‌شود تا نشان بدهد که پردازه‌ی فرزند، دستگیره‌های مشخص شده را از والد خود به ارث می‌برد. پیش از نوشتن لوله، والد در ابتدا طرف خواندن بلااستفاده‌ی لوله‌ی خود را می‌بندد. پردازه‌ی فرزند که از لوله می‌خواند در شکل ۳-۲۵ نشان داده می‌شود. این برنامه پیش از خواندن لوله، دستگیره‌ی خواندن لوله را با احضار GetStdHandle() به‌دست می‌آورد.

```
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#define BUFFER_SIZE 25
#define READ_END      0
#define WRITE_END 1
int main(void) {
        char write_msg[BUFFER_SIZE] = "Greetings";
        char read_msg[BUFFER_SIZE];
        int fd[2];
        pid_t pid;

        /* create the pipe */
        if (pipe(fd) == -1) {
                fprintf(stderr,"Pipe failed");
                return 1;
        }
        /* fork a child process */
        pid = fork();
        if (pid < 0) { /* error occurred */
                fprintf(stderr, "Fork Failed");
                return 1;
        }
        if (pid > 0) {  /* parent process */
                /* close the unused end of the pipe */
                close(fd[READ_END]);
                /* write to the pipe */
```

/* ادامه‌ی برنامه در شکل ۳-۲۲ */

شکل ۳-۲۱ لوله‌های معمولی در یونیکس.

```
                write(fd[WRITE_END], write_msg, strlen(write_msg)+1);
                /* close the write end of the pipe */
                close(fd[WRITE_END]);
        }
        else { /* child process */
                /* close the unused end of the pipe */
                close(fd[WRITE_END]);
                /* read from the pipe */
                read(fd[READ_END], read_msg, BUFFER_SIZE);
                printf("read %s\n",read_msg);

                /* close the read end of the pipe */
                close(fd[READ_END]);
        }
        return 0;
}
```

شکل ۳-۲۲ ادامه‌ی برنامه‌ی شکل ۳-۲۱.

توجه داشته باشید که لوله‌های معمولی در هر دوی سیستم‌های یونیکس و ویندوز مستلزم وجود رابطه‌ی والد-فرزندی میان پردازه‌های ارتباطی می‌باشند. این بدان معنی است که این لوله‌ها فقط برای ارتباط میان پردازه‌های روی یک ماشین یکسان قابل استفاده هستند.

۳-۷-۴-۲ لوله‌های با نام

لوله‌های معمولی راهکار ساده‌ای برای امکان ارتباط میان یک جفت پردازه فراهم می‌کنند. با این وجود، لوله‌های معمولی فقط زمانی وجود دارند که پردازه‌ها با همدیگر در ارتباط هستند. در هر دوی سیستم‌های یونیکس و ویندوز، همین که پردازه‌ها ارتباط را به پایان رسانده و خاتمه بپذیرند، لوله‌ی معمولی متوقف می‌شوند.

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
#define BUFFER_SIZE 25
int main(VOID) {
 HANDLE ReadHandle, WriteHandle;
 STARTUPINFO si;
 PROCESS_INFORMATION pi;
 char message[BUFFER_SIZE] = "Greetings";
 DWORD written;
/* set up security attributes allowing pipes to be inherited */
 SECURITY_ATTRIBUTES sa = {sizeof(SECURITY_ATTRIBUTES), NULL,TRUE};
```

/* ادامه‌ی برنامه در شکل ۳-۲۴ */

شکل ۳-۲۳ لوله بی‌نام ویندوز—پردازه‌ی والد.

```
 /* allocate memory */
 ZeroMemory(&pi, sizeof(pi));

 /* create the pipe */
 if ( !CreatePipe(&ReadHandle, &WriteHandle, &sa, 0)) {
    fprintf(stderr,"Create Pipe Failed");
    return 1;
 }
 /* establish the START_INFO structure for the child process */
 GetStartupInfo(&si);
 si.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);

 /* redirect standard input to the read end of the pipe */
 si.hStdInput = ReadHandle;
 si.dwFlags = STARTF_USESTDHANDLES;
 /* don't allow the child to inherit the write end of pipe */
 SetHandleInformation(WriteHandle, HANDLE_FLAG_INHERIT, 0);

 /* create the child process */
 CreateProcess(NULL, "child.exe", NULL, NULL,
       TRUE, /* inherit handles */
       0, NULL, NULL, &si, &pi));
 /* close the unused end of the pipe */
 CloseHandle(ReadHandle);
/* the parent writes to the pipe */
if (!WriteFile (WriteHandle, message, BUFFER_SIZE, &written, NULL))
    fprintf(stderr, "Error writing to pipe.");
 /* close the write end of the pipe */
 CloseHandle(WriteHandle);
 /* wait for the child to exit */
 WaitForSingleObject(pi.hProcess, INFINITE);
 CloseHandle(pi.hProcess);
 CloseHandle(pi.hThread);
 return 0;
}
```

شکل ۳-۲۴ ادامه‌ی شکل ۳-۲۳.

لوله‌های با نام یک ابزار ارتباطی پرقدرتی فراهم می‌کنند. ارتباط می‌تواند دو طرفه بوده و رابطه‌ی والد-فرزندی هم لازم نیست. همین که یک لوله‌ی با نام برپا شد، چندین پردازه می‌توانند برای ارتباط از آن استفاده کنند. در حقیقت، در شرایط معمولی، یک لوله‌ی با نام می‌تواند چندین نویسنده داشته باشد. به علاوه، لوله‌های با نام حتی با پایان‌یافتن پردازه‌های ارتباطی می‌توانند وجود داشته باشند. هر دوی سیستم‌های یونیکس و ویندوز از لوله‌های با نام پشتیبانی می‌کنند،

هرچند که جزئیات پیاده‌سازی آنها تفاوت زیادی با هم دارند. در ادامه، لوله‌های با نام هر یک از این سیستم‌ها را مرور می‌کنیم.

```
#include <stdio.h>
#include <windows.h>
#define BUFFER_SIZE 25
int main(VOID)
{
    HANDLE ReadHandle;
    CHAR buffer[BUFFER_SIZE];
    DWORD read;
    /* get the read handle of the pipe */
    ReadHandle = GetStdHandle(STD_INPUT_HANDLE);
    /* the childs reads from the pipe */
    if (ReadFile(ReadHandle, buffer, BUFFER_SIZE, &read, NULL))
       printf("child read %s",buffer);
    else
        fprintf(stderr, "Error reading from pipe");
    return 0;
}
```

شکل ۳-۲۵ لوله‌های بی‌نام ویندوز—پردازه‌ی فرزند.

لوله‌های با نام سیستم‌های ویندوز در مقایسه با همتاهای خود در یونیکس، راهکار ارتباطی غنی‌تری فراهم می‌نمایند. ارتباط دوطرفه‌ی کامل ممکن بوده و پردازه‌های مرتبط می‌توانند مقیم یک ماشین و یا متفاوت باشند. به علاوه، در FIFOهای یونیکس فقط امکان انتقال داده‌های بایتی وجود دارد، حال آنکه در سیستم‌های ویندوز، امکان انتقال داده‌های بایتی و یا پیغامی وجود دارد. لوله‌های با نام توسط تابع CreateNamedPipe() ایجاد شده و کارفرما می‌تواند با استفاده از ConnectNamedPipe() به یک لوله‌ی با نام متصل شود. ارتباط روی لوله‌ی با نام می‌تواند با استفاده از توابع ReadFile() و WriteFile() انجام شود.

لوله‌ها در عمل

لوله‌ها اغلب در محیط خط فرمان یونیکس در شرایطی که خروجی یک فرمان به عنوان ورودی فرمان دیگر استفاده می‌شود، به‌کار می‌آیند. برای مثال، فرمان ls یونیکس، فهرست یک دایرکتوری را نمایش می‌دهد. اگر فهرست یک دایرکتوری خاصی طولانی باشد امکان دارد که خروجی فرمان به چندین صفحه گسترش یابد. فرمان more، خروجی را با نمایش صفحه به صفحه مدیریت می‌نماید؛ کاربر بایستی کلید space را فشار دهد تا به صفحه‌ی دیگر جابه‌جا شود. با برپایی لوله‌ای میان فرمان‌های ls و more (که به عنوان پردازه‌هایی مجزا اجرا می‌شوند)، این امکان به وجود می‌آید که خروجی ls به عنوان ورودی برای more تحویل گردد تا کاربر بتواند فهرست یک دایرکتوری طولانی را صفحه به صفحه نمایش بدهد. یک لوله در خط فرمان می‌تواند با استفاده از کاراکتر | برپا شود. فرمان کامل آن به صورت زیر خواهد بود: